

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Спроектировать класс согласно варианту индивидуального задания. Для класса использовать отдельный модуль.

1. Предусмотреть у класса наличие полей, методов и свойств. Названия членов класса должны быть осмысленны.
2. Для всех нетекстовых полей разработать свойства, контролирующие корректность значения (например, оценка – от 0 до 10). Одно из свойств (имя объекта) должно быть только для чтения с инициализацией в конструкторе. Поля, связанные со свойствами, должны быть инкапсулированы.
3. Разработать несколько конструкторов класса (не менее 3). Один – без аргументов, с заполнением полей значением по умолчанию, второй – с полным набором аргументов и третий с частичным набором аргументов. Связать конструкторы в цепочку, дабы избежать дублирования кода. Для проверки корректности аргументов в конструкторах использовать свойство.
4. Переопределить метод ToString() для отображения всех полей класса.
5. Разработать указанные для вашего варианта методы.
6. Создать автоматическую диаграмму спроектированного класса.
7. Создать несколько объектов класса. Продемонстрировать использование всех конструкторов.
8. Продемонстрировать работу всех методов.
9. Продемонстрировать использование всех свойств – для получения и присваивания значения.
10. Создать еще одну переменную, присвоив ей один из предыдущих объектов.
11. Продемонстрировать, что две переменные ссылаются на один объект (поменять поле в одном объекте и вывести его в другом).

Вариант 8. Класс «Магазин».

Поля:

- a. название магазина,
- b. количестве продавцов,
- c. суммарной выручке,
- d. средней зарплате,
- e. стоимости закупки,
- f. накладных расходах

Методы:

- a. ToString(),
- b. метод для определения премии (чистая прибыль за месяц/ к-во продавцов)
- c. метод, определяющий, падение рентабельности (если прибыль меньше 10% от суммарной выручки)
- d. метод, определяющий более рентабельную фирму из двух – по проценту

рентабельности (возвращает true, если текущая более рентабельна)
е. статический метод, определяющий более рентабельную фирму из трех
(возвращает объект, чья рентабельность выше)

ВЫПОЛНЕНИЕ ЗАДАНИЯ

Листинг программы приведен в приложении А.

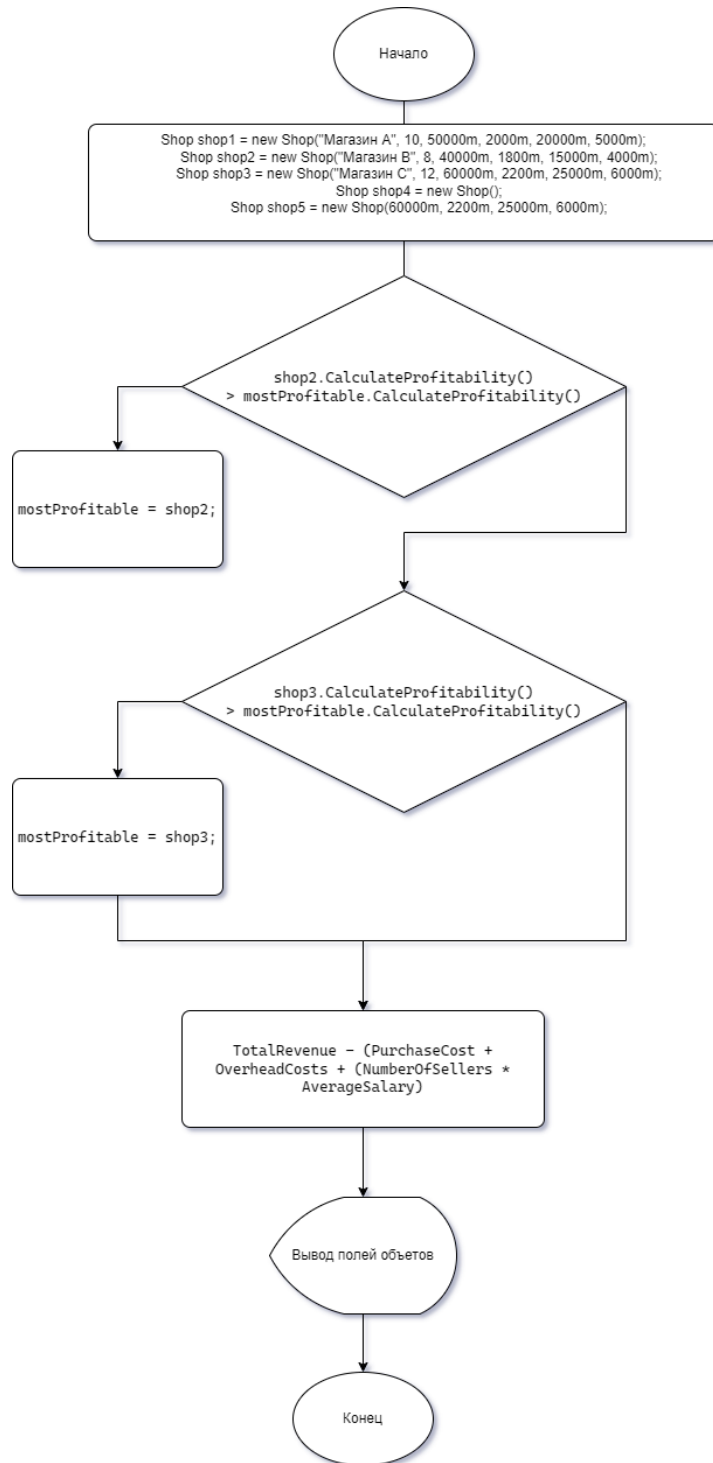


Рисунок 1.1 – Блок-схема

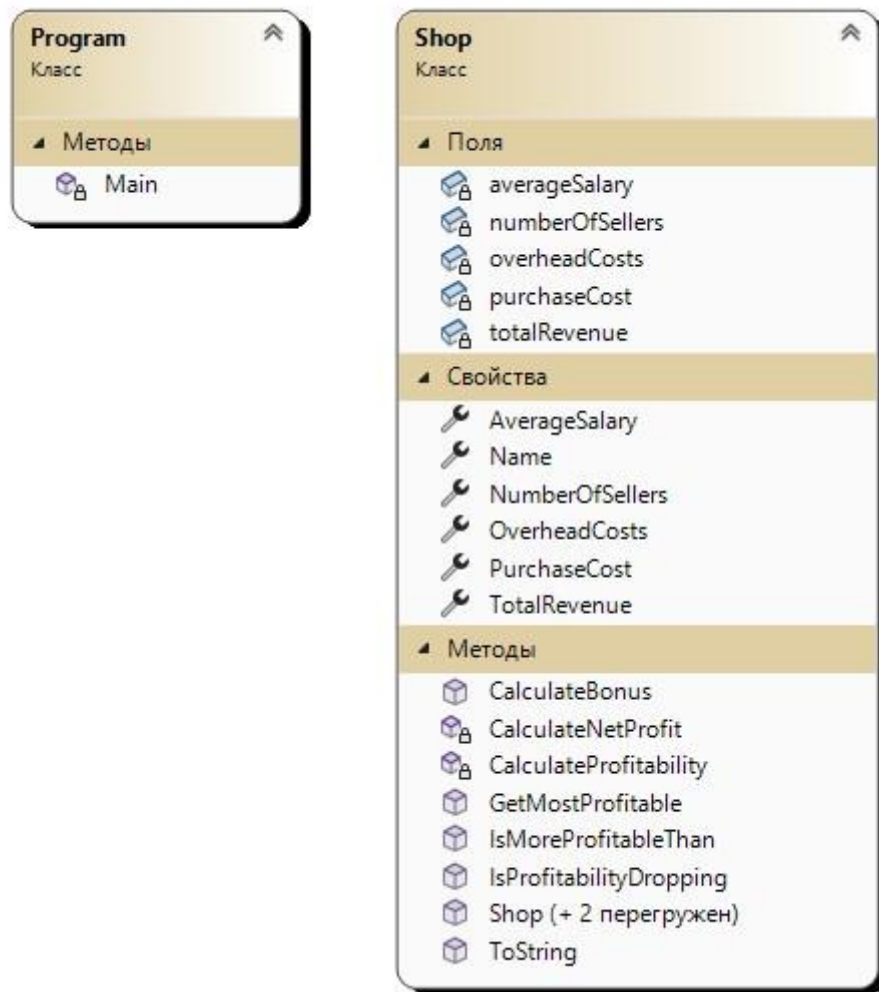


Рисунок 1.2 – Диаграмма классов

Результат выполнения программы (снимок экрана):

The screenshot shows the Visual Studio IDE with the following components:

- Code Editor:** Displays the source code for `Program.cs`. The code includes logic for creating three `Shop` objects, calculating their bonuses, and determining the most profitable one.
- Output Window:** Shows the console output of the program, including the names of the shops, their sales, and the calculated bonuses.
- Task List:** Displays a list of tasks or errors, including a warning about a nullable reference type.
- Properties Window:** Shows the properties of the selected object, including its name and various calculated values.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Положение об организации и проведении курсового проектирования в БГУИР. – Введ. 2013-12-11. – Минск: БГУИР, 2013. – 18 с.
2. Библиографическая запись. Сокращение слов на русском языке : общие требования и правила: ГОСТ 7.0.12–2011. – М.: Стандартинформ, 2012. – 17 с.
3. Библиографическая запись. Сокращение слов и словосочетаний на иностранных европейских языках : ГОСТ 7.11–2004 (ИСО832:1994). – Минск: Госстандарт Респ. Беларусь : Белорус. гос. ин-т стандартизации и сертификации, 2005. – 83 с.
4. СТП 01–2017. Дипломные проекты (работы): общие требования. – Введ. 2018. – Утв. с изм. 24.11.2017. – Минск: БГУИР, 2017. – 169 с.
5. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : ГОСТ 7.1–2003. – Минск: Госстандарт Респ. Беларусь, 2004. – 48 с.
6. ГОСТ 19.701-90 Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения, реализовано программное средство, удовлетворяющее поставленной задаче, построена блок-схема алгоритма, диаграмма классов.

ПРИЛОЖЕНИЕ А
(справочное)
Листинг программы

```
using System;
class Shop
{
    // Поля
    int numberOfSellers;
    decimal totalRevenue;
    decimal averageSalary;
    decimal purchaseCost;
    decimal overheadCosts;

    // Свойства
    public string Name { get; }
    public int NumberOfSellers
    {
        get => numberOfSellers;
        set => numberOfSellers = Math.Abs(value);
    }
    public decimal TotalRevenue
    {
        get => totalRevenue;
        set => totalRevenue = Math.Abs(value);
    }
    public decimal AverageSalary
    {
        get => averageSalary;
        set => averageSalary = Math.Abs(value);
    }
    public decimal PurchaseCost
    {
        get => purchaseCost;
        set => purchaseCost = Math.Abs(value);
    }
    public decimal OverheadCosts
    {
        get => overheadCosts;
        set => overheadCosts = Math.Abs(value);
    }

    // Конструкторы
    public Shop()
    {
        Name = "Магаз";
        NumberOfSellers = 3;
        TotalRevenue = 40000m;
        AverageSalary = 1800m;
        PurchaseCost = 15000m;
        OverheadCosts = 4000m;
    }
    public Shop(decimal totalRevenue, decimal averageSalary, decimal purchaseCost,
decimal overheadCosts) : this()
    {
        TotalRevenue = totalRevenue;
        AverageSalary = averageSalary;
        PurchaseCost = purchaseCost;
        OverheadCosts = overheadCosts;
    }
}
```



```

    public Shop(string name, int numberOfSellers, decimal totalRevenue, decimal
averageSalary, decimal purchaseCost, decimal overheadCosts) : this(totalRevenue,
averageSalary, purchaseCost, overheadCosts)
    {
        Name = name;
        NumberOfSellers = numberOfSellers;
    }

    // Методы
    public override string ToString() => $"Название: {Name}, Продавцов:
{NumberOfSellers}, Суммарная выручка: {TotalRevenue}, Средняя з/п: {AverageSalary},
Стоимость закупки: {PurchaseCost}, Накладные расходы: {OverheadCosts}";

    // Премия
    public decimal CalculateBonus() => CalculateNetProfit() / NumberOfSellers;

    // Падение рентабельности
    public bool IsProfitabilityDropping() => (CalculateNetProfit() / TotalRevenue) <
0.10m;

    // Более рентабельная фирма из двух
    public bool IsMoreProfitableThan(Shop other) => CalculateProfitability() >
other.CalculateProfitability();

    // Более рентабельная фирма из трех
    public static Shop GetMostProfitable(Shop shop1, Shop shop2, Shop shop3)
    {
        Shop mostProfitable = shop1;
        if (shop2.CalculateProfitability() >
mostProfitable.CalculateProfitability()) {
            mostProfitable = shop2;
        }
        if (shop3.CalculateProfitability() >
mostProfitable.CalculateProfitability()) {
            mostProfitable = shop3;
        }
        return mostProfitable;
    }

    // Расчет прибыли
    private decimal CalculateNetProfit() => TotalRevenue - (PurchaseCost +
OverheadCosts + (NumberOfSellers * AverageSalary));
    private decimal CalculateProfitability() => CalculateNetProfit() / TotalRevenue;
}
class Program
{
    static void Main()
    {
        Shop shop1 = new Shop("Магазин А", 10, 50000m, 2000m, 20000m, 5000m);
        Shop shop2 = new Shop("Магазин В", 8, 40000m, 1800m, 15000m, 4000m);
        Shop shop3 = new Shop("Магазин С", 12, 60000m, 2200m, 25000m, 6000m);
        Shop shop4 = new Shop();
        Shop shop5 = new Shop(60000m, 2200m, 25000m, 6000m);

        Console.WriteLine(shop4.ToString());
        shop4.NumberOfSellers = -33;
        Console.WriteLine(shop4.ToString());

        Console.WriteLine(shop1.ToString());
        Console.WriteLine($"Премия: {shop1.CalculateBonus()}");
        Console.WriteLine($"Снижается ли рентабельность:
{shop1.IsProfitabilityDropping()}");

        Console.WriteLine(shop2.ToString());

```

```

        Console.WriteLine($"Премия: {shop2.CalculateBonus()}");
        Console.WriteLine($"Снижается ли рентабельность:
{shop2.IsProfitabilityDropping()}");

        Console.WriteLine(shop3.ToString());
        Console.WriteLine($"Премия: {shop3.CalculateBonus()}");
        Console.WriteLine($"Снижается ли рентабельность:
{shop3.IsProfitabilityDropping()}");

        Console.WriteLine($"Фирма {shop1.Name} более рентабельна, чем {shop2.Name}:
{shop1.IsMoreProfitableThan(shop2)}");

        Shop mostProfitable = Shop.GetMostProfitable(shop1, shop2, shop3);
        Console.WriteLine($"Более рентабельная фирма из {shop1.Name}, {shop2.Name},
{shop3.Name}: {mostProfitable.Name}");

        Console.ReadKey();
    }
}

```